实验 3：谓词逻辑与 SMT 求解

1. 实验主题

使用 Z3 建立有类型的 SMT 模型，使用谓词和量词表达一般规则，并通过模型、反例、不可满足核和优化结果回答软件工程问题。

2. 实验目标

- 理解布尔可满足性问题（Boolean satisfiability problem，SAT）与可满足性模理论（satisfiability modulo theories，SMT）的差别，并正确使用布尔型（Bool）、整数型（Int）和位向量（BitVec）。
- 使用 Z3Py 把自然语言规则写成有类型的约束，从模型中还原并检查具体方案。
- 使用谓词、全称量词（universal quantifier）和存在量词（existential quantifier）表达一般规则，并通过加入性质的否定检查结论是否成立。
- 理解数学整数与机器整数的边界语义，使用不可满足核（unsatisfiable core，简称 unsat core）解释需求冲突，并区分可满足方案与最优方案。

允许使用 LLM 协助编写候选约束或解释 API，但必须通过 Z3 运行、边界实例、反例或第二实现核验。如果使用 LLM 生成正式约束或代码，实验报告还应包含实际提示词、你最终采用或修改的内容和独立核验结果。

3. 工具与参考资料

- Z3 Guide：Z3Py Introduction
- Programming Z3
- Z3 官方代码库

4. 最小环境检查（Smoke Test）

4.1 Z3Py

保存为 `z3_smoke.py`：

```
from z3 import Int, Solver

x = Int("x")
s = Solver()
s.add(x > 1, x < 3)

assert str(s.check()) == "sat"
assert s.model()[x].as_long() == 2
print("Z3 smoke test passed")
```

运行：

```
python3 z3_smoke.py
```

环境正确时，应输出 `Z3 smoke test passed`。若导入失败，应先检查当前 Python 环境中安装的是 `z3-solver` 包，而不是同名的其他包。

5. 实验任务

任务 1：同一部署场景的 CNF 与 SMT 表示

任务摘要： 对同一个软件部署问题分别建立布尔 CNF 和整数 SMT 模型，比较两种表示所需的变量、约束和领域解释。

某系统包含四个组件和三个部署节点：

组件	记号
网关（Gateway）	`G`
认证服务（Authentication）	`A`
数据库（Database）	`D`
缓存（Cache）	`C`

节点编号为 `0、1、2`，每个组件必须部署到且仅部署到一个节点。另有以下规则：

1. 网关与数据库不能部署在同一节点。
2. 认证服务与数据库必须部署在同一节点。
3. 缓存不能部署在节点 0。
4. 网关只能部署在节点 0 或节点 1。
5. 如果缓存部署在节点 1，则网关必须部署在节点 0。

完成以下工作：

1. 建立布尔变量 `X_组件_节点`，其含义为“该组件部署在该节点”。为 4 个组件和 3 个节点建立共 12 个变量，把“每个组件恰好位于一个节点”和上述五条规则全部写成 CNF 子句，并使用 Z3Py 的 `Bool`、`Or` 和 `Not` 求出一个模型。
2. 为四个组件分别建立一个取值范围为 `0..2` 的 `Int` 变量，用整数相等、不等、取值范围和 `Implies` 直接写出同一组规则，并求出一个模型。
3. 比较两种表示使用的变量数量、约束形式和可读性，并选择“认证服务与数据库同节点”这条规则，完整展示它在 CNF 和整数 SMT 中的两种写法。

实验报告应包含： 固定场景的变量表、完整 CNF 子句、整数 SMT 约束、两个 Z3 模型及其部署方案，以及两种表示的变量数量、约束形式和指定规则写法的比较；能够看清两次求解状态和模型的关键截图；以及你对 CNF 与 SMT 表达差异自行撰写的结论。

任务 2（编程题）：检查课程冲突排课方案

任务摘要： 实现一个 C++ 函数，判断给定排课方案是否满足时间段范围和全部课程冲突约束。

一般地，设有 `n` 门课程和 `k` 个时间段：

- 课程编号为 `0..n-1`。
- 时间段编号为 `0..k-1`。

- `slot` 包含 `n` 个元素，`slot[i]` 表示课程 `i` 使用的时间段。
- 冲突对 `(u,v)` 表示课程 `u` 与课程 `v` 不能使用同一时间段。

检查函数需要适用于任意符合接口约定的 `n`、`k` 和冲突列表。本任务使用下面的具体实例设计测试：某专业需要把六门课程安排到三个时间段，由同一位教师授课或选课名单有重叠的课程不能同时上课。本任务不考虑教室和课程容量。

- 本例课程编号为 `0..5`。
- 本例时间段编号为 `0..2`。

本任务使用的冲突关系为：

```
(0,1), (0,2), (1,3), (2,3), (2,4), (3,5), (4,5)
```

例如，`slot[0]=0, slot[1]=0` 违反冲突 `(0,1)`；当前冲突列表中没有 `(0,3)`，所以 `slot[0]=0, slot[3]=0` 本身不违反规则。

请在 `lab3_solution.cpp` 中实现：

```cpp
#include <utility>
#include <vector>

bool validSchedule(
    int slotCount,
    const std::vector<int>& slot,
    const std::vector<std::pair<int, int>>& conflicts) {
    // 在此处完成实现
}
```

接口约定：

- 合法时间段编号为 `0` 到 `slotCount - 1`，其中 `1 <= slotCount <= 100`。
- `conflicts` 中的课程下标均合法；每对冲突课程必须位于不同时间段。
- 不在冲突列表中的课程可以位于同一时间段。
- 函数不得定义 `main`，不得进行标准输入输出；评分程序将使用 C++17 编译并直接调用该函数。

1. 完成 `validSchedule`，依次检查每门课程的时间段编号是否有效，以及每对冲突课程是否被安排在不同时间段。
2. 自行构造并运行四类测试：合法方案、时间段编号越界、冲突课程位于同一时间段、非冲突课程位于同一时间段且方案合法。
3. 从步骤 2 的合法测试中找出一对位于同一时间段但彼此不冲突的课程。用这对课程说明函数为什么只比较冲突课程，而不能要求所有课程的时间段都互不相同。

实验报告应包含： 最终 C++ 代码、四类测试的具体输入和实际返回值、两类检查与排课规则的对应关系，以及合法测试中共享时间段的一对非冲突课程；能够看清编译和测试结果的关键截图；以及你对“怎样判断一个给定排课方案是否合法”自行撰写的结论。

任务 3：使用 Z3 求解课程冲突排课问题

任务摘要： 把课程时间段和冲突规则写成整数约束，让 Z3 自动寻找合法方案，并观察新增冲突如何改变求解结果。

某专业需要把六门课程安排到若干时间段。课程编号为 `0..5`，`slot[i]` 表示课程 `i` 所在的时间段；冲突对 `(u,v)` 表示两门课程不能同时上课，即 `slot[u] != slot[v]`。本任务使用以下冲突关系：

```
(0,1), (0,2), (1,3), (2,3), (2,4), (3,5), (4,5)
```

1. 为六门课程分别建立一个 Z3 `Int` 变量，加入时间段范围 `0..2` 和上述七条冲突约束，求出一个使用三个时间段的合法方案。把模型转换为任务 2 中 `validSchedule` 所需的 `slot`，调用该函数检查方案，并按时间段列出课程。
2. 在冲突关系中增加 `(0,3)`。分别把所有课程的时间段限制在 `0..1` 和 `0..2`，记录两个求解结果。根据完整冲突列表找出一个足以说明两个时间段不可行的最小课程集合，并用该集合解释两个结果为什么不同。

实验报告应包含： Z3 变量、时间段范围和七条冲突约束、原问题的三时间段方案及 `validSchedule` 检查结果、增加 `(0,3)` 后使用两个和三个时间段的求解结果，以及说明两个时间段不可行的最小课程集合；能够看清求解状态、模型和检查函数结果的关键截图；以及你对“约束求解器怎样从规则中寻找排课方案”自行撰写的结论。

任务 4：比较 Int 与 8 位 BitVec 的加法语义

任务摘要： 使用三种数值语义检查同一个性质“`x+1` 是否一定大于 `x`”，理解数学整数、无符号位向量和有符号位向量在边界处为什么得到不同结果。

Z3 的 `Int` 表示没有固定位宽的数学整数。`BitVec("x", 8)` 表示 8 位位向量，加法结果只保留低 8 位。同一个 8 位结果可以按无符号数或有符号数解释：`UGT(a, b)` 进行无符号大于比较，`a > b` 对位向量进行有符号大于比较。

本任务检查性质：

```
x + 1 > x
```

为了让 Z3 寻找该性质的反例，应向求解器加入它的否定，即“加一后的值不大于原值”。保存以下无符号版本为 `unsigned_increment.py`：

```
from z3 import *

x = BitVec("x", 8)
y = x + 1

s = Solver()
s.add(Not(UGT(y, x)))
print(s.check())
print(s.model())
```

1. 运行无符号版本，记录反例中的 `x`，计算对应的 `y`，并根据 8 位无符号数的取值范围解释为什么“加一后更大”在该输入上不成立。
2. 保持 `x`、`y` 和 8 位加法不变，把待检查性质改为有符号比较，使用 `Not(y > x)` 寻找反例。记录反例，并按 8 位有符号数解释 `x` 和 `y`。
3. 重新建立一个类型为 Z3 `Int` 的变量，用数学整数加法重写同一个性质“加一后不大于原值”，运行求解器并解释为什么结果与两个 `BitVec` 版本不同。
4. 建立对照表，分别列出数学整数、8 位无符号位向量和 8 位有符号位向量的取值范围、比较方式、求解结果、反例和边界处的加法结果。

实验报告应包含： 三种模型及运行命令、各自的求解结果、两个 `BitVec` 反例的数值解释、三种语义的对照表；能够看清反例和 `Int` 求解状态的关键截图；以及你对“变量类型为什么会改变验证结论”自行撰写的结论。

任务 5：使用谓词和量词检查模块发布条件

任务摘要： 使用谓词表示软件模块的测试与发布状态，使用全称量词表达所有模块共同遵守的规则，并使用存在量词和性质否定检查结论。

谓词（predicate）用于表示对象是否具有某种性质。本任务包含 `Parser`、`Database` 和 `UI` 三个软件模块，并定义两个一元谓词：`Tested(x)` 表示模块 `x` 已经通过测试，`Releasable(x)` 表示模块 `x` 可以发布。下面已经给出模块类型、测试状态事实和两条发布规则组成的基础模型：

```
from z3 import *

Module, (Parser, Database, UI) = EnumSort(
    "Module", ["Parser", "Database", "UI"])
Tested = Function("Tested", Module, BoolSort())
Releasable = Function("Releasable", Module, BoolSort())

m = Const("m", Module)

base_constraints = [
    Tested(Parser),
    Not(Tested(Database)),
    Tested(UI),
    ForAll([m], Implies(Tested(m), Releasable(m))),
    ForAll([m], Implies(Releasable(m), Tested(m)))
]

base_solver = Solver()
base_solver.add(base_constraints)
print(base_solver.check())

model = base_solver.model()
for module in [Parser, Database, UI]:
    print(module,
          model.eval(Tested(module)),
          model.eval(Releasable(module)))
```

其中，`Parser` 和 `UI` 已经通过测试，`Database` 尚未通过测试。两个 `ForAll` 规则表示：

- 已经通过测试的模块可以发布；
- 可以发布的模块一定已经通过测试。

该基础模型可满足，并且两条规则共同要求每个模块的 `Tested` 与 `Releasable` 取值一致。在此基础上完成以下工作：

1. 分别检查以下三个性质是否由事实和规则推出：存在至少一个可以发布的模块；`Database` 不可以发布；所有模块都可以发布。分别使用存在量词（existential quantifier）`Exists`、逻辑否定 `Not` 和 `ForAll` 表示这三个性质。每次建立独立检查，在基础约束中加入待检查性质的否定：结果为 `unsat` 表示性质必然成立，结果为 `sat` 表示模型给出了不满足该性质的情况。
2. 将基础事实中的“`Database` 尚未通过测试”替换为“`Database` 已经通过测试”，再次检查“所有模块都可以发布”。比较修改前后的结果，并解释一个事实的变化如何通过全称规则改变三个模块的发布状态。

实验报告应包含： 模块枚举类型、两个谓词、测试状态事实、两条全称量词规则和三个待检查性质；各次检查的 `sat/unsat` 结果及其含义；修改 `Database` 测试状态前后的对照；能够看清求解状态和三个模块关键谓词取值的截图；以及你对“如何使用性质的否定判断结论是否由规则推出”自行撰写的结论。

任务 6：使用不可满足核解释服务部署冲突

任务摘要： 在给定的服务部署模型上提取不可满足核，把核中的标签还原为容量冲突，并通过删除约束检查这个核是否还能缩小。

不可满足核（unsatisfiable core）是一组已经足以导致无解的带标签约束。求解器返回的核不一定是所有可能冲突中约束数量最少的一组，因此还需要通过删减约束继续检查。

某系统准备在维护期间部署 API、任务处理器和数据库。API 至少需要两个副本，每个 API 副本都需要一个任务处理器副本，数据库至少需要两个副本，系统还必须预留一个备用节点。每个服务副本和备用节点都独占一个节点，而维护期间最多只有六个节点可用。系统另外要求启用监控，并规定启用监控时必须启用审计。保存以下模型为 `deployment_core.py`：

```
from z3 import *

api_replicas = Int("api_replicas")
worker_replicas = Int("worker_replicas")
database_replicas = Int("database_replicas")
standby_nodes = Int("standby_nodes")
total_nodes = Int("total_nodes")
monitoring = Bool("monitoring")
audit = Bool("audit")

domain_constraints = [
    api_replicas >= 0,
    worker_replicas >= 0,
    database_replicas >= 0,
    standby_nodes >= 0,
    total_nodes >= 0
]

requirements = [
    ("api_high_availability", api_replicas >= 2),
    ("worker_matches_api", worker_replicas >= api_replicas),
```

```
    ("database_high_availability", database_replicas >= 2),
    ("standby_required", standby_nodes >= 1),
    ("one_service_per_node",
     total_nodes == api_replicas + worker_replicas
                    + database_replicas + standby_nodes),
    ("maintenance_capacity", total_nodes <= 6),
    ("monitoring_required", monitoring),
    ("monitoring_requires_audit", Implies(monitoring, audit))
]

solver = Solver()
solver.add(domain_constraints)
for name, constraint in requirements:
    solver.assert_and_track(constraint, Bool(name))

print(solver.check())
print(solver.unsat_core())
```

其中，`domain_constraints` 只规定变量的基本取值范围；`requirements` 中的八条带标签约束对应需要分析的部署要求。

1. 运行给定模型，记录求解结果和 `unsat_core()` 返回的全部标签。
2. 把核中的每个标签还原为自然语言要求，根据副本数、节点计算关系和维护期间的节点上限，说明这些要求为什么不能同时满足；同时说明哪些带标签要求没有出现在核中。
3. 对核中的每条约束分别做一次删减检查：每次重新建立求解器，保留领域约束和其余部署要求，删除当前这一条后重新求解，记录结果并判断当前核是否已经不能继续删减。另外选择一条监控或审计约束单独删除并求解，比较它与删除容量冲突约束后的结果。

实验报告应包含： 给定模型的运行结果、不可满足核、标签与自然语言要求的对应、容量冲突的计算说明、每条核心约束的删减结果，以及删除一条非核心约束后的结果；能够看清不可满足核和删减检查结果的关键截图；以及你对“不可满足核中的约束与其它约束有什么区别”自行撰写的结论。

任务 7：求解使用时间段最少的排课方案

任务摘要： 在固定课程冲突关系上建立优化目标，求出使用时间段最少的排课方案，并解释新增冲突为什么改变最优值。

需要将六门课程安排到至多三个候选时间段。课程编号为 `0..5`，时间段编号为 `0..2`。每门课程必须选择一个时间段，下面每对存在冲突的课程不能使用同一时间段：

```
(0,1), (0,2), (1,3), (2,3), (2,4), (3,5), (4,5)
```

完成以下工作：

1. 使用六个 Z3 `Int` 变量建立上述排课模型，并确认在三个候选时间段内存在合法方案。
2. 为三个时间段分别建立布尔变量 `used[t]`，加入 `used[t] == Or(slot[i] == t)`，使其准确表示“至少一门课程使用时间段 `t`”；使用 `Sum(If(used[t], 1, 0))` 表示实际使用的时间段数。
3. 使用 Z3 `Optimize` 最小化实际使用的时间段数，记录原冲突关系下的最优值和一个最优方案，并逐条检查全部冲突。

4. 在原冲突关系中增加一条新的冲突 `(1,2)` 后重新优化，比较修改前后的最少时间段数。根据修改后的冲突列表找出一个能够给出新最优值下界的课程集合，再用求得的方案说明该下界可以达到。

实验报告应包含： 完整排课约束、`used[t]` 的定义、优化目标、原冲突关系和新增 `(1,2)` 后的最优值与最优方案、新最优值的下界课程集合，以及逐条冲突检查；能够看清两个优化结果的关键截图；以及你对“可满足方案与最优方案有什么区别”自行撰写的结论。

6. 评分标准

本实验总分为 100 分。编程题根据自动测试结果和报告中的测试分析评分，其余任务根据模型、运行结果、截图和分析评分。

任务	分值
任务 1：同一部署场景的 CNF 与 SMT 表示	15
任务 2：检查课程冲突排课方案	10
任务 3：使用 Z3 求解课程冲突排课问题	15
任务 4：比较 Int 与 8 位 BitVec 的加法语义	15
任务 5：使用谓词和量词检查模块发布条件	15
任务 6：使用不可满足核解释服务部署冲突	15
任务 7：求解使用时间段最少的排课方案	15
总分	**100**